RAG 문서 준비 교안

문서를 검색할 수 있도록 준비하기

보고서를 글 조각으로 나누기

문서를 검색할 수 있도록 준비하고 저장하기

보고서를 글 조각으로 나누고, 숫자로 바꾸고, ChromaDB에 함께 저장하기까지

이 실습의 Python 프로그램은 회사 보고서를 검색할 글 조각으로 나눕니다. 임베딩 모델은 각 조각을 숫자 목록으로 바꿉니다. 프로그램은 글과 숫자, 출처 정보를 같은 ID로 연결해 ChromaDB에 저장합니다. **이번 교안은 이 준비 과정을 다룹니다.**



'청크'는 검색하기 좋게 나눈 작은 글 조각입니다. 이 교안의 주 사례는 2025년 3분기 보고서의 매출액 표입니다.

범위 이번 교안은 질문에 답하는 문장을 만들지 않습니다. 검색에 쓸 자료를 준비하고 저장하는 과정까지입니다.

다음 → 이 교안을 읽고 나면 무엇을 설명할 수 있어야 하는지 봅니다

문서를 검색할 수 있도록 준비하고 저장하기

보고서를 글 조각으로 나누고, 숫자로 바꾸고, ChromaDB에 함께 저장하기까지

이 실습의 Python 프로그램은 회사 보고서를 검색할 글 조각으로 나눕니다. 임베딩 모델은 각 조각을 숫자 목록으로 바꿉니다. 프로그램은 글과 숫자, 출처 정보를 같은 ID로 연결해 ChromaDB에 저장합니다. **이번 교안은 이 준비 과정을 다룹니다.**



'청크'는 검색하기 좋게 나눈 작은 글 조각입니다. 이 교안의 주 사례는 2025년 3분기 보고서의 매출액 표입니다.

한국폴리텍대학교 하이테크 2026-2 · RAG 문서 준비·저장 교안 · 2026-09-21 · 실습 자료 rag_exam(NB01·NB02)

범위 이번 교안은 질문에 답하는 문장을 만들지 않습니다. 검색에 쓸 자료를 준비하고 저장하는 과정까지입니다.

다음 → 이 교안을 읽고 나면 무엇을 설명할 수 있어야 하는지 봅니다

이 교안을 읽고 설명할 수 있어야 하는 일곱 가지

외울 용어 목록이 아니라, 학생이 혼자 설명할 수 있어야 할 내용입니다. 마지막 활동의 문항과 이어집니다.

① 어떤 파일을 읽으며, 문서를 왜 나누는지 설명합니다.

② 조각에서 기간·단위·열 제목이 빠지면 어떤 문제가 생기는지 설명합니다.

③ 700자 분할과 앞 설명·머리글 보강의 차이를 설명합니다.

④ 청크 본문, ID, 출처 정보, 임베딩 숫자의 역할을 구분합니다.

⑤ 실제 코드와 결과를 보고 파일·DB에 무엇이 저장되는지 확인합니다.

⑥ 재실행 시 어느 자료가 지워지고 다시 만들어지는지 설명합니다.

⑦ 다음 검색 노트북이 어떤 파일·설정을 읽는지 설명합니다.

활동과 연결

①~⑦은 마지막 활동 1~7번과 이어집니다. 답과 이유는 활동 다음 화면에 있습니다.

이 교안은 NB01(문서를 청크로 나누기)과 NB02(임베딩과 ChromaDB 저장)의 실제 코드와 결과 화면을 순서대로 읽습니다. 설명을 위해 일부 코드를 먼저 소개해도 노트북은 위에서 아래로 실행합니다.

기준 ‘설명할 수 있다’가 기준입니다. 숫자를 외우는 것이 목표가 아닙니다.

다음 → 사용자가 실제로 넣는 질문과 보고서의 표를 먼저 봅니다

사용할 보고서와 질문

사용자는 2025년 3분기 3개월 연결 매출액을 알고 싶어 합니다. 보고서에는 3개월 값과 누적 값이 같은 행에 있습니다.

사용자의 질문 “2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?”

원문 일부 재구성

제공 원문 000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md · III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서

연결 포괄손익계산서 · 제 78 기 3분기 2025.01.01 부터 2025.09.30 까지 · (단위 : 백만원)

	제 78 기 3분기 3개월	제 78 기 3분기 누적	제 77 기 3분기 3개월	제 77 기 3분기 누적
매출액 (주4,21,28)	24,448,929	64,320,022	17,573,069	46,425,925

원문 표의 매출액 행에는 열이 네 개 있습니다. 제 78 기(2025년) 3개월 값 24,448,929와 누적 값 64,320,022, 제 77 기(2024년)의 같은 두 값입니다. **프로그램이 글을 나눌 때 이 값들을 구분할 열 제목과 단위를 조각에 남겨야 합니다.**

표는 제공된 실습용 원문의 연결 포괄손익계산서에서 제목·기간·단위·열 제목·매출액 행만 골라 다시 구성했습니다. 외부 최신 공시를 새로 확인한 값이 아닙니다.

표 옆의 말뜻

공시 회사가 공개한 보고 자료

연결 회사와 종속기업을 함께 집계한 기준

3분기 3개월 2025년 7~9월

누적 2025년 1~9월

백만원 숫자 1 = 1,000,000원

확인

두 값 중 하나가 틀린 값은 아닙니다. 질문이 3개월을 물으므로 사용자는 3개월 열을 읽습니다. 표 위의 보고 기간만 보고 모든 열을 누적으로 읽지 않습니다.

다음 → 이 표가 어떤 파일과 프로그램을 거쳐 저장되는지 순서를 봅니다

파일과 프로그램이 이어지는 순서

NB01의 프로그램은 MD 파일을 읽고 chunks.jsonl을 만듭니다. NB02의 프로그램은 이 파일을 다시 읽어 임베딩과 DB 저장을 진행합니다. NB02는 다음 검색 프로그램이 읽을 index.json도 저장합니다.

입력 · MD 파일 5개

rag_exam/data 폴더. 2025FY, 2025Q3, 2026H1, 2026Q1, 시설투자 공시

NB01 · 01_문서를_청크로_나누기.ipynb

프로그램이 파일을 읽고, 정리하고, 제목·문단·길이로 나눕니다

결과 · artifacts/chunks.jsonl

청크 800개. 한 줄에 청크 한 개 (id·source·period·section·text)

NB02 · 02_Ollama_임베딩과_ChromaDB_저장.ipynb

파일을 읽고, 모델이 숫자를 만들고, DB에 저장합니다

결과 · chroma_db + artifacts/index.json

컬렉션 skhynix_basic에 글·숫자·출처 저장. 설정 파일에 값 4개

메모리 목록과 디스크 파일은 다릅니다

NB01의 chunks 변수는 커널을 끝내면 사라집니다. NB02는 NB01의 변수를 자동으로 받지 않고 chunks.jsonl 파일을 읽습니다.

다음 검색 프로그램이 읽는 것

NB03(03_ChromaDB에서_근거_조회.ipynb)은 chroma_db와 index.json을 읽어 질문에 가까운 후보를 찾습니다. 이 교안은 NB03 실행을 다루지 않습니다.

순서 노트북은 위에서 아래로 실행합니다. 이 교안은 설명을 위해 길이 설정(코드 10)을 앞 설명(코드 8)보다 먼저 소개하지만 실행 순서는 바꾸지 않습니다.

다음 → NB01의 첫 코드부터 봅니다. 프로그램은 어느 폴더에서 파일을 찾습니까?

프로그램이 파일을 찾을 폴더를 정합니다

프로그램은 현재 작업 폴더 안의 data와 artifacts를 가리키는 이름을 만들고, 현재 작업 폴더의 전체 위치를 화면에 보여 줍니다. 파일 안의 글은 아직 읽지 않습니다.

실행 결과

[1]:
```
# 폴더와 파일의 위치를 다루는 Path 도구를 가져옵니다. import는 준비된 도구를 불러온다는 뜻입니다.
from pathlib import Path

# 현재 작업 폴더 안의 data를 가리킵니다. =는 오른쪽 값을 왼쪽 이름에 담으라는 뜻입니다.
data_folder = Path("data")
# 프로그램이 만든 청크·설정·테스트 결과를 둘 위치입니다. 이 줄만으로 폴더가 생기지는 않습니다.
output_folder = Path("artifacts")
# 현재 작업 폴더의 전체 경로를 반환합니다. 마지막 줄의 값은 Jupyter가 화면에 보여 줍니다.
Path.cwd()
```

NB01 코드 1 · 실행 출력

결과 data_folder, output_folder라는 이름과 현재 폴더 표시. 이 셀은 파일 안의 글을 읽지 않습니다. **확인** 표시된 위치가 실습 폴더인지 봅니다. 화면의 위치는 실습 환경에 따라 다를 수 있습니다.

주의 Path("data")는 현재 작업 폴더 안의 data라는 뜻입니다. 현재 위치가 다르면 같은 코드가 다른 폴더를 가리킵니다.

다음 → data 폴더에서 어떤 파일을 찾는지 봅니다

data 폴더의 MD 파일 목록을 만듭니다

프로그램은 data 폴더에서 이름이 .md로 끝나는 파일을 찾아 이름순으로 정렬한 목록(files)을 만듭니다. 이 단계에서도 파일 안의 글은 읽지 않습니다.

실행 결과

[2]:
```
# MD 파일 위치만 모아 이름순으로 정렬합니다. 아직 파일 안의 글은 읽지 않았습니다.
files = sorted(data_folder.glob("*.md"))
# 찾은 파일 목록을 화면에서 확인합니다.
files
```

[2]:
```
[WindowsPath('data/000660_SK하이닉스_2025FY_원문발췌_실습용_v1.0.md'),
 WindowsPath('data/000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md'),
 WindowsPath('data/000660_SK하이닉스_2026H1_원문발췌_실습용_v1.0.md'),
 WindowsPath('data/000660_SK하이닉스_2026Q1_원문발췌_실습용_v1.1.md'),
 WindowsPath('data/000660_SK하이닉스_신규시설투자등_20260722800829_실습용_v1.0.md')]
```

NB01 코드 2 · 실행 출력

결과 MD 파일 5개의 경로 목록(files) **제외** 같은 폴더의 질문·정답 CSV는 .md가 아니므로 목록에 없습니다. **순서** 이름순 정렬이므로 2025FY 파일이 첫 번째입니다.

확인 목록이 비어 있으면 프로그램이 다른 폴더에서 찾고 있는 것입니다. 코드 1의 현재 폴더부터 확인합니다.

다음 → 파일 안의 글을 실제로 읽는 코드를 봅니다

파일 안의 글을 문자열로 읽어 목록에 담습니다

프로그램은 목록의 파일을 하나씩 열어 read_text로 글 전체를 문자열로 읽습니다. 파일명(source)과 글(text)을 함께 documents 목록에 담습니다.

실행 결과

[3]:
```python
# 읽은 문서들을 담을 빈 목록입니다. 이후 append가 문서 한 개씩 추가합니다.
documents = []
# files의 첫 번째 파일부터 마지막 파일까지 차례로 처리합니다.
for path in files:
    # 파일 내용을 글자로 읽습니다. utf-8-sig는 파일 앞의 숨은 문자 표시가 있어도 처리합니다.
    text = path.read_text(encoding="utf-8-sig")
    # 파일명(source)과 전체 본문(text)을 함께 보관합니다. 원본 파일은 수정하지 않습니다.
    documents.append({"source": path.name, "text": text})
```

NB01 코드 3 · 실행 출력

결과 documents: {"source": 파일명, "text": 글} 5개 **범위** 이 코드는 MD 문자열을 읽습니다. PDF나 HWP를 직접 분석하지 않습니다. **화면 출력 없음** 값은 메모리의 documents에 남습니다.

이유 나누기 도구는 파일 이름이 아니라 글을 받아야 합니다. 파일명을 함께 두어야 검색 뒤에 원문을 찾을 수 있습니다.

다음 → 파일명에서 보고 기간을 정하는 코드를 봅니다

파일명에서 보고 기간을 정해 period에 넣습니다

프로그램은 파일명을 _ 기준으로 나눈 세 번째 조각을 period로 씁니다. 시설투자 공시는 파일명에 '신규시설투자등'이 있으면 별도 조건으로 20260722를 붙입니다.

실행 결과

[4]:
```python
# 문서마다 기간 이름표를 하나씩 붙입니다.
for document in documents:
    # 딕셔너리에서 source라는 이름에 담긴 파일명을 꺼냅니다.
    filename = document["source"]
    # 파일명에 이 글자가 들어 있으면 날짜로 구분하는 투자 공시입니다.
    if "신규시설투자등" in filename:
        # 해당 공시의 날짜를 period라는 새 항목으로 추가합니다.
        document["period"] = "20260722"
    # 투자 공시가 아닌 일반 보고서는 아래 규칙을 사용합니다.
    else:
        # 목록 번호는 0부터 시작하므로 [2]는 세 번째 조각입니다.
        document["period"] = filename.split("_")[2]
```

NB01 코드 4 · 실행 출력

확인할 점

규칙 000660_SK하이닉스_2025Q3_... → 2025Q3

예외 '신규시설투자등' 파일 → 20260722

결과 문서마다 period 항목 추가

주의 이 실습의 파일명에 맞춘 규칙입니다. 질문에서 기간을 자동으로 읽는 기능이 아닙니다.

이유

같은 회사라도 연간·분기·반기 금액이 다릅니다. 다음 검색에서 사용자가 고른 기간의 문서만 찾으려면 기간 정보가 필요합니다.

다음 → 읽어 둔 글에서 줄바꿈 표시와 정렬선을 정리합니다

글자 표시 정리 ①
와 긴 정렬선

프로그램은 메모리에 읽어 둔 글에서
 표시를 공백으로 바꾸고, ---가 길게 이어진 정렬선을 ---로 줄입니다. 원문 MD 파일은 바꾸지 않고 내용을 요약하지도 않습니다.

실행 결과

[5]:
```python
# 일정한 글자 모양을 찾아 바꾸는 re 도구를 불러옵니다.
import re

# 원본 파일이 아니라 메모리에 읽어 둔 문서별 text를 정리합니다.
for document in documents:
    # 예: 3분기<br>3개월을 3분기 3개월로 바꿉니다.
    text = document["text"].replace("<br>", " ")
    # 표의 긴 정렬선을 ---로 통일합니다. 매출액 숫자는 대상이 아닙니다.
    text = re.sub(r"-{3,}", "---", text)
```

확인할 점

정리 전(원문 548행) 제 78 기 3분기
3개월

정리 후 제 78 기 3분기 3개월

대상 document["text"] (메모리)

원문 MD 파일 그대로 보존

같은 셀 코드 5의 1~9행. 10행부터 이어집니다.

표시 정리 전후 예는 원문 548행 기준

구분
는 표 안의 줄바꿈 표시입니다. 정리하면 열 제목이 한 줄로 읽히지만 표의 숫자나 단위는 바뀌지 않습니다.

다음 → 같은 셀의 뒷부분에서 줄마다 공백을 정리하고 다시 합칩니다

글자 표시 정리 ② 줄마다 공백을 정리해 다시 합칩니다

프로그램은 글을 한 줄씩 나누어 연속 공백을 하나로 줄이고 양끝 공백을 없앤 뒤, 줄바꿈으로 다시 이어 document["text"]에 넣습니다.

실행 결과

```
    # 정리한 줄들을 담을 빈 목록입니다.
    lines = []
    # 전체 글을 한 줄씩 꺼내 처리합니다.
    for line in text.splitlines():
        # 연속 공백·탭을 공백 하나로 바꾸고 줄의 양끝 공백을 없앱니다.
        line = re.sub(r"[ \t]+", " ", line).strip()
        # 정리한 한 줄을 순서대로 모읍니다.
        lines.append(line)
    # 모은 줄 사이에 줄바꿈을 넣어 다시 문서 전체 글로 만듭니다.
    document["text"] = "\n".join(lines)
```

앞페이지와 같은 셀에 구현

확인할 점

입력 정리 중인 text

처리 splitlines() → 줄마다 re.sub와 strip() → "\n".join

결과 정리된 글이 같은 문서의 text에 다시 들어갑니다.

같은 셀 코드 5의 10~19행

이유 정렬용 공백과 표시는 글의 뜻보다 보기 방식에 가깝습니다. 정리하면 불필요한 글자 수가 줄어듭니다.

다음 → 제목 표시로 장과 절을 나누는 도구를 준비합니다

##와 ###를 읽는 제목 분리 도구를 준비합니다

프로그램은 MarkdownHeaderTextSplitter에 ##는 chapter, ###는 section이라는 이름으로 나누도록 설정합니다. 이 셀은 설정만 하고 아직 나누지 않습니다.

실행 결과

[6]:
```python
# MD 제목을 읽어 본문과 제목 정보를 나누는 도구를 가져옵니다.
from langchain_text_splitters import MarkdownHeaderTextSplitter

# 제목별 분리를 맡을 도구에 설정을 전달합니다.
header_splitter = MarkdownHeaderTextSplitter(
    # ##로 시작한 제목은 chapter, ###로 시작한 제목은 section이라는 이름으로 보관합니다.
    headers_to_split_on=[("##", "chapter"), ("###", "section")]
)
```

NB01 코드 6 · 실행 출력

설정 [("##", "chapter"), ("###", "section")] **원문 예** ## III. 재무에 관한 사항 / ### 2-2. 연결 포괄손익계산서 **범위** # 하나짜리 문서 제목은 이 설정에 없으므로 본문으로 남습니다.

구분 '분리 도구'는 프로그램이 정한 규칙으로 글을 나누는 도구입니다. 제목의 뜻을 판단하는 인공지능 기능이 아닙니다.

다음 → 도구로 문서를 나누고 장·절 제목을 한 줄로 연결합니다

절별로 나누고 '장 > 절' 제목을 만듭니다

프로그램은 문서마다 제목 분리 도구를 적용해 절 단위로 나눕니다. 각 부분의 장·절 제목을 ' > '로 이어 heading 한 줄을 만듭니다.

실행 결과

```
[7]: # 제목별로 나눈 글을 모을 빈 목록입니다.
sections = []
# 파일별로 분리해야 다른 문서의 제목이나 출처가 섞이지 않습니다.
for document in documents:
    # 이제 실제 문서 본문을 도구에 넣어 제목별로 나눕니다.
    divided = header_splitter.split_text(document["text"])
    # 도구가 돌려준 본문 조각을 하나씩 꺼냅니다.
    for part in divided:
        # 장 제목과 절 제목을 사람이 읽을 수 있는 한 줄로 연결합니다.
        heading = " > ".join(part.metadata.values())
```

NB01 코드 7 · 1~10행 · 실행 출력

확인할 점

입력 documents의 text

처리 header_splitter.split_text → 부분(part) 목록

제목 예 III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서

파일별 처리 다른 문서의 제목이 섞이지 않도록 문서마다 따로 나눕니다.

이유 매출액 표가 어느 장·절에 있는지 알아야 숫자의 뜻을 확인할 수 있습니다. 제목은 버리지 않고 정보로 남깁니다.

다음 → 제목이 없는 앞부분을 어떻게 표시하는지 봅니다

제목이 없으면 '공시 개요', 절마다 네 정보를 남깁니다

프로그램은 제목이 빈 부분에 '공시 개요'를 붙이고, 절마다 파일명·기간·제목·본문(page_content)을 sections 목록에 넣습니다.

실행 결과

```
        # ==는 같은지 비교합니다. 제목이 빈 글이면 아래 이름을 사용합니다.
        if heading == "":
            # 문서 맨 앞의 제목 없는 안내도 출처를 붙여 보관합니다.
            heading = "공시 개요"
        # 본문 조각마다 출처·기간·제목·본문 네 가지를 함께 담습니다.
        sections.append({
            # 파일명과 기간은 원래 문서의 값을 그대로 이어받습니다.
            "source": document["source"], "period": document["period"],
            # page_content는 도구가 나눈 본문 글입니다.
            "section": heading, "text": part.page_content
        })
```

앞페이지와 같은 셀에 구현

확인할 점

조건 heading == "" 이면 "공시 개요"

예 2025FY 문서 맨 앞의 '# 사업보고서' 부분

결과 sections: source, period, section, text

본문 page_content는 제목 줄을 뺀 글입니다.

확인 첫 청크 2025FY_0001의 section이 '공시 개요'인 이유가 이 조건입니다.

다음 → 나누기 전에, 숫자만 남은 조각에서 무엇이 빠지는지 봅니다

숫자만 남은 조각에서 빠진 정보

표에 24,448,929와 64,320,022만 남으면 사용자는 어느 값이 3개월 매출액인지 알기 어렵습니다. 열 제목과 단위가 함께 있어야 질문에 맞는 값을 고를 수 있습니다.

설명용 실패 예

| 매출액 (주4,21,28) | 24,448,929 | 64,320,022 | 17,573,069 | 46,425,925 |

제목·기간·단위·열 제목을 의도적으로 덜어낸 예입니다. 현재 분할 코드가 만든 실제 결과가 아닙니다.

저장된 청크 2025Q3_0196의 앞부분(chunks.jsonl)

SK하이닉스 2025Q3 / III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서 / | 연결 포괄손익계산서 | ... | (단위 : 백만원) | / | | 제 78 기 3분기 3개월 | 제 78 기 3분기 누적 | ... | / | 매출액 (주4,21,28) | 24,448,929 | 64,320,022 | ... |

활동

이 활동에서는 학생이 두 조각의 차이를 확인합니다. 왼쪽 조각에서 빠진 정보는 무엇이며, 그 정보가 없으면 어떤 값을 고르기 어렵습니까?

해설

빠진 것은 열 제목(3개월·누적), 단위(백만원), 기간·절 제목입니다. 열 제목이 없으면 3개월 값과 누적 값을 구분할 수 없고, 단위가 없으면 금액의 크기를 잘못 읽을 수 있습니다.

목표 프로그램은 작은 조각을 만들면서 이런 설명도 함께 남겨야 합니다. 다음 코드들은 이 설명을 남기는 방법입니다.

다음 → 먼저 본문을 나누는 길이 설정을 봅니다

본문을 700자 기준으로 나누고 200자를 겹치게 설정합니다

분할 도구는 본문을 700자 기준으로 나눕니다. 도구는 줄바꿈, 마침표와 공백, 공백, 글자 순서로 나눌 자리를 찾습니다.
chunk_overlap=200은 이웃 본문의 일부가 겹치도록 하는 설정입니다.

실행 결과

[10]:
```python
# 길이를 보면서 여러 구분자를 차례로 사용하는 분리 도구입니다.
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 실제 본문을 넣기 전에 나누는 규칙부터 준비합니다.
splitter = RecursiveCharacterTextSplitter(
    # 줄바꿈 → 마침표와 공백 → 공백 → 글자 순서로 나눌 자리를 찾습니다.
    chunk_size=700, chunk_overlap=200, separators=["\n", ". ", " ", ""]
)
```

NB01 코드 10 · 실행 출력

확인할 점

700 본문을 나누는 글자 수 설정. 토큰 수가 아닙니다.

200 겹침의 목표 설정. 모든 조각이 정확히 200자씩 겹친다는 보장은 아닙니다.

구분자 ["\n", ". ", " ", ""] 순서

최종 길이 프로그램이 뒤에서 제목·머리글을 붙이므로 최종 청크는 700자보다 길 수 있습니다.

표시 설명 순서상 먼저 소개. 실행은 코드 8·9 뒤입니다.

구분

앞 문단 보강(코드 8)과 길이 분할의 겹침은 다른 처리입니다. 겹침 설정만으로 표의 단위와 열 제목이 항상 남는 것은 아닙니다.

다음 → 표 앞의 짧은 설명을 함께 남기는 코드(코드 8)를 봅니다

절의 글을 문단으로 나누고 앞 두 문단을 살펴봅니다

프로그램은 절의 글을 제목 분리 도구가 문단 사이에 남긴 '공백 두 칸과 줄바꿈' 표시로 문단으로 나눕니다. 각 문단에 대해 바로 앞의 최대 두 문단을 previous에 가져옵니다.

실행 결과

[8]:
```python
# 문단과 주변 설명을 묶어 담을 목록입니다.
blocks = []
# 제목별 본문 하나씩 처리합니다.
for section in sections:
    # 제목 분리 도구가 넣은 공백 두 칸과 줄바꿈을 기준으로 문단을 나눕니다.
    paragraphs = section["text"].split("  \n")
    # 문단 번호를 0부터 끝까지 순서대로 사용합니다.
    for number in range(len(paragraphs)):
        # 현재 문단 바로 앞의 짧은 설명을 모읍니다.
        context = []
        # 현재 문단은 제외하고 최대 앞 두 문단만 가져옵니다.
        previous = paragraphs[max(0, number - 2):number]
```

NB01 코드 8 · 1~12행 · 실행 출력

확인할 점

문단 구분
section["text"].split(" \n")

범위 paragraphs[max(0, number - 2):number] → 앞 최대 두 문단

첫 문단 앞 문단이 없으면 빈 목록입니다.

이 화면 코드 8의 1~12행

구분

'최대 두 문단'은 현재 문단 바로 앞에서 살펴볼 범위입니다. 문서 전체에서 가장 알맞은 설명을 찾는 기능이 아닙니다.

다음 → 그중 어떤 문단을 앞 설명으로 붙이는지 조건을 봅니다

250자 미만인 앞 문단만 context에 모읍니다

프로그램은 가져온 앞 문단 중 길이가 250자 미만인 문단만 context에 넣습니다. 현재 문단과 context, 출처 정보를 blocks의 한 항목으로 만듭니다.

실행 결과

```
        # 가져온 앞 문단을 하나씩 확인합니다.
        for paragraph in previous:
            # 긴 본문 전체가 중복되지 않도록 250글자 미만인 설명만 고릅니다.
            if len(paragraph) < 250:
                # 기간이나 단위처럼 짧은 앞 설명을 보관합니다.
                context.append(paragraph)
        # 현재 문단과 그 문단의 이름표·앞 설명을 한 항목으로 만듭니다.
        blocks.append({
            "source": section["source"], "period": section["period"],
            # context 목록을 줄바꿈으로 연결해 하나의 글로 저장합니다.
            "section": section["section"], "context": "\n".join(context),
            # number번째 문단 자체는 text에 넣습니다.
            "text": paragraphs[number]
        })
```

NB01 코드 8 · 13~26행 · 실행 출력 앞페이지와 같은 셀에 구현

확인할 점

조건 if len(paragraph) < 250 ('이하'가 아니라 '미만')

예 표 앞의 '(단위 : 백만원)' 줄이 든 짧은 문단

결과 blocks: source, period, section, context, text

한계 프로그램이 단위 문장의 뜻을 알아보고 고르는 것은 아닙니다. 길이 조건으로 고릅니다.

확인 context 전체가 250자 미만이라는 뜻이 아닙니다. 문단 하나하나가 250자 미만이면 최대 두 개까지 들어갑니다.

다음 → 표로 보이는 문단의 첫 두 줄을 따로 보관하는 코드를 봅니다

문단이 표인지 확인할 준비를 합니다

프로그램은 문단을 줄 단위로 나눕니다. 우선 모든 문단의 header를 빈 글로, body를 문단 전체로 둡니다. 표가 아니면 이 값이 그대로 남습니다.

실행 결과

```
[9]: # 문단 묶음마다 표인지 확인합니다.
for block in blocks:
    # 현재 문단을 줄 단위 목록으로 바꿉니다.
    lines = block["text"].splitlines()
    # 우선 일반 문단이라고 보고 머리글을 빈 글로 둡니다.
    block["header"] = ""
    # 표가 아니라면 본문은 현재 글 전체입니다.
    block["body"] = block["text"]
```

NB01 코드 9 · 1~8행 · 실행 출력

확인할 점

입력 blocks의 text

기본값 header = "", body = 문단 전체

표일 때만 뒤 조건에서 header와 body를 다시 나눕니다.

이 화면 코드 9의 1~8행

구분 일반 문단은 머리글 없이 본문만 나뉩니다. 표만 첫 두 줄을 따로 보관합니다.

다음 → 표를 판별하는 모양 조건을 봅니다

둘째 줄이 표 구분선이면 첫 두 줄을 header에 보관합니다

프로그램은 줄이 세 개 이상이고 둘째 줄이 |, 공백, :, -로만 이루어졌으며 |를 포함하면 표로 봅니다. 그때 첫 두 줄을 header에, 나머지 줄을 body에 둡니다.

실행 결과

```
    # 머리글 두 줄과 본문이 있어야 이 방식으로 표를 나눌 수 있습니다.
    if len(lines) > 2:
        # 둘째 줄 전체가 표 구분선에 쓰이는 글자들로만 구성되었는지 확인합니다.
        separator = re.fullmatch(r"[| :\-]+", lines[1])
        # 모양이 맞고 세로 구분선 |도 있어야 표로 취급합니다.
        if separator and "|" in lines[1]:
            # 처음 두 줄을 머리글로 보관하고 끝에 줄바꿈을 붙입니다.
            block["header"] = "\n".join(lines[:2]) + "\n"
            # 나머지 행만 길이 기준으로 나눌 본문에 둡니다.
            block["body"] = "\n".join(lines[2:])
```

NB01 코드 9 · 9~18행 · 실행 출력 앞페이지와 같은 셀에 구현

확인할 점

첫 두 줄 예 '| | 제 78 기 3분기 3개월 | 제 78 기 3분기 누적 | ... |'와 '| --- | --- | ... |'

body 매출액 행부터의 본문 행

뒤 단계 프로그램은 body를 나눈 뒤 각 조각 앞에 header를 다시 붙입니다(코드 11).

한계 표 전체나 긴 한 행을 절대 자르지 않는다는 보장은 없습니다.

주의 header에는 열 제목 줄과 구분선 줄이 들어갑니다. 표 위의 '(단위 : 백만원)' 줄은 별도 문단이라 앞 설명(context) 쪽에 들어갈 수 있습니다.

다음 → 본문을 나눈 뒤 최종 글을 조합하는 코드를 봅니다

청크 목록과 파일별 번호 세기를 준비합니다

프로그램은 완성한 청크를 담을 chunks 목록과 파일별로 몇 번째 청크인지 세는 counts를 만듭니다. 블록마다 출처와 기간은 앞 단계의 값을 그대로 씁니다.

실행 결과

[11]:

```
# 완성한 청크들을 담을 목록입니다.
chunks = []
# 각 출처 파일을 몇 번째 청크까지 만들었는지 기록합니다.
counts = {}
# 문단 묶음마다 본문을 나누고 이름표를 붙입니다.
for block in blocks:
    # 출처와 기간은 새로 추측하지 않고 앞 단계에서 가져옵니다.
    source = block["source"]
    period = block["period"]
    # 처음 만난 파일은 번호 세기를 0에서 시작합니다.
    if source not in counts:
        counts[source] = 0
```

NB01 코드 11 · 1~12행 · 실행 출력

확인할 점

chunks 최종 청크 목록(처음엔 비어 있음)

counts {파일명: 지금까지 만든 청크 수}

처음 보는 파일 counts[source] = 0부터 시작

이 화면 코드 11의 1~12행. 같은 셀이 13행부터 이어집니다.

이유 청크 ID는 기간과 파일별 순번으로 만듭니다(예: 2025Q3_0196). 순번을 세려면 파일별 개수가 필요합니다.

다음 → 본문을 나누고 제목·앞 설명·머리글을 붙이는 부분을 봅니다

본문을 나눈 뒤 회사·기간·절·앞 설명·머리글을 붙입니다

분할 도구는 body를 700자 기준의 조각(pieces)으로 나눕니다. 프로그램은 조각마다 'SK하이닉스 기간', 절 제목, context를 앞에 쓰고, 그 뒤에 header와 조각(piece)을 이어 최종 text를 만듭니다.

실행 결과

```
    # 이곳에서 실제 본문을 최대 700글자 규칙으로 나눕니다.
    pieces = splitter.split_text(block["body"])
    # 나누어진 본문 조각 하나가 최종 청크 하나가 됩니다.
    for piece in pieces:
        # 현재 파일의 청크 번호를 1 증가시킵니다. += 1은 기존 값에 1을 더한다는 뜻입니다.
        counts[source] += 1
        # 회사·기간·장과 절·앞 설명을 본문 앞에 붙일 글로 만듭니다.
        text = f"SK하이닉스 {period}\n{block['section']}\n{block['context']}\n"
        # 보관한 표 머리글과 이번 본문 조각을 이어 붙입니다.
        text = text + block["header"] + piece
```

NB01 코드 11 · 13~22행 · 실행 출력 앞페이지와 같은 셀에 구현

결합 순서 회사·기간 → 절 제목 → context → header → piece **빈 값** context나 header가 비어 있으면 그 자리는 빈 줄로 남습니다. **결과** 최종 text. 700자보다 길 수 있습니다(이번 실행 28~1,055자). **같은 셀** 코드 11의 13~22행

구분 700자 분할(splitter)과 정보 보강(text 결합)은 다른 처리입니다. 최종 글이 700자를 넘는 것은 오류가 아니라 보강의 결과입니다.

다음 → 완성한 글에 ID와 출처 정보를 붙이는 부분을 봅니다

청크 하나에 id, source, period, section, text를 기록합니다

프로그램은 완성한 글에 id, source, period, section을 함께 기록합니다. id는 기간과 파일별 순번(네 자리)으로 만든 번호이고, text에는 앞 단계에서 만든 글이 들어 있습니다.

실행 결과

```
        # 완성한 글과 검색용 이름표를 같은 청크에 넣습니다.
        chunks.append({
            # 예: 2025Q3_0196. :04d는 번호를 네 자리로 표시합니다.
            "id": f"{period}_{counts[source]:04d}", "source": source,
            # 검색 기간 조건, 원문 위치, 실제 검색 본문을 함께 보관합니다.
            "period": period, "section": block["section"], "text": text
        })
```

NB01 코드 11 · 23~29행 · 실행 출력 앞페이지와 같은 셀에 구현

저장된 청크 2025Q3_0196의 다섯 필드

id 2025Q3_0196

source 000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md

period 2025Q3

section III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서

text SK하이닉스 2025Q3 … | 매출액 (주4,21,28) | 24,448,929 | … (910자)

주의 id는 현재 파일별 생성 순서에 따른 번호입니다. 원문 페이지 번호나 영구히 변하지 않는 주소가 아닙니다. 문서형식·발표일 같은 필드는 없습니다.

다음 → 만든 청크의 첫 항목과 파일별 개수를 확인합니다

첫 청크를 눈으로 확인합니다

chunks[0]에서 실제 첫 청크를 확인합니다. 이번 실행의 첫 청크는 2025FY_0001이며 section은 '공시 개요'입니다.

신규 실행 출력

```
[12]: # 목록의 첫 청크를 화면에 보여 줍니다. [0]은 0개가 아니라 첫 번째 항목입니다.
      chunks[0]
```

```
[12]: {'id': '2025FY_0001',
       'source': '000660_SK하이닉스_2025FY_원문발췌_실습용_v1.0.md',
       'period': '2025FY',
       'section': '공시 개요',
       'text': 'SK하이닉스 2025FY\n공시 개요\n\n# 사업보고서'}
```

NB01 코드 12 · 실행 출력

확인할 점

첫 청크 이름순 첫 파일(2025FY)의 첫 조각

text 'SK하이닉스 2025FY', '공시 개요', 빈 줄, '# 사업보고서'

주 사례와 구분 Q3 매출액 표는 2025Q3_0196입니다. 첫 출력에는 매출액이 보이지 않습니다.

확인 새 실행에서 첫 청크가 다르면 파일 정렬·입력 파일명·제목 처리를 확인합니다. 과거 숫자로 출력 화면을 고치지 않습니다.

다음 → 파일별 청크 개수를 확인합니다

파일별 청크 개수를 확인합니다

counts에서 각 파일의 청크 수를 확인합니다. 이번 실행의 다섯 파일 합계는 800개입니다. 이 숫자는 아직 메모리 목록의 개수이며 파일 저장은 다음 셀입니다.

신규 실행 출력

```
[13]: # 각 파일을 키, 그 파일의 청크 개수를 값으로 가진 딕셔너리를 보여 줍니다.
      counts
```

```
[13]: {'000660_SK하이닉스_2025FY_원문발췌_실습용_v1.0.md': 202,
       '000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md': 203,
       '000660_SK하이닉스_2026H1_원문발췌_실습용_v1.0.md': 199,
       '000660_SK하이닉스_2026Q1_원문발췌_실습용_v1.1.md': 188,
       '000660_SK하이닉스_신규시설투자등_20260722800829_실습용_v1.0.md': 8}
```

NB01 코드 13 · 실행 출력

확인할 점

2025FY 202 · **2025Q3** 203 · **2026H1** 199 · **2026Q1** 188 · **시설투자** 8

합계 800

뜻 counts는 파일명별 개수입니다. 값을 외우는 것이 목표가 아닙니다.

확인 새 실행에서 800이 아니면 MD 목록·입력 파일·라이브러리 버전·실행 순서를 확인합니다. 숫자를 맞추려고 코드를 고치지 않습니다.

다음 → 청크를 파일로 저장하는 코드를 봅니다

청크 한 개를 JSON 한 줄로 바꿔 chunks.jsonl에 씁니다

프로그램은 artifacts 폴더를 만들고(이미 있으면 그대로), 청크 하나를 JSON 형태의 글 한 줄로 바꿔 chunks.jsonl에 차례로 씁니다. JSONL은 '한 줄에 청크 한 개를 기록한 파일'입니다.

실행 결과

[14]:
```python
# 딕셔너리를 파일에 쓸 수 있는 JSON 글로 바꾸는 도구입니다.
import json

# artifacts가 없으면 만듭니다. exist_ok=True라서 이미 있어도 그대로 사용합니다.
output_folder.mkdir(exist_ok=True)
# 결과 파일을 쓰기 모드로 열고 file이라는 이름으로 사용합니다.
with (output_folder / "chunks.jsonl").open("w", encoding="utf-8") as file:
    # 청크를 순서대로 하나씩 꺼냅니다.
    for chunk in chunks:
        # 현재 청크 한 개를 한 줄짜리 JSON 글로 바꿉니다.
        line = json.dumps(chunk, ensure_ascii=False)
        # 청크 한 줄을 기록하고 다음 청크를 위해 줄을 바꿉니다.
        file.write(line + "\n")
```

NB01 코드 14 · 실행 출력

해당 실행 결과 · JSONL 한 줄

한 줄 예(2025FY_0001) {"id": "2025FY_0001", "source": "...", "period": "2025FY", "section": "공시 개요", "text": "SK하이닉스 2025FY\n공시 개요\n\n# 사업보고서"}

\n 표시 text 안의 줄바꿈은 파일에서 \n 두 글자로 표시됩니다.

덮어쓰기 "w" 모드는 기존 파일을 새로 씁니다.

이유 다음 노트북은 이 노트북의 메모리 변수를 공유하지 않습니다. 청크를 파일로 남겨야 NB02가 다시 읽을 수 있습니다.

다음 → NB02가 이 파일을 다시 읽는 코드를 봅니다

NB02는 chunks.jsonl을 읽어 청크 목록을 다시 만듭니다

NB02의 프로그램은 같은 작업 폴더의 artifacts/chunks.jsonl을 한 줄씩 읽고, 각 줄을 JSON에서 딕셔너리로 되돌려 chunks 목록에 넣습니다. NB02 코드 1은 NB01 코드 1과 같은 코드로 폴더를 정합니다.

실행 결과

[2]:
```python
# 파일 속 JSON 글을 딕셔너리로 되돌리는 도구입니다.
import json

# 파일에서 다시 읽은 청크들을 담을 목록입니다.
chunks = []
# 01이 저장한 chunks.jsonl을 읽기 모드로 엽니다.
with (output_folder / "chunks.jsonl").open(encoding="utf-8") as file:
    # 800줄을 한 줄씩 읽습니다. 한 줄은 청크 한 개입니다.
    for line in file:
        # 그 줄의 JSON 글을 딕셔너리로 바꿔 목록에 넣습니다.
        chunks.append(json.loads(line))
```

NB02 코드 2 · 실행 출력

확인할 점

입력 chunks.jsonl 800줄

결과 NB02의 chunks 목록 800개

NB02 코드 1 NB01 코드 1과 같은 코드

주의 파일이 없으면 NB01이 먼저 저장했는지, 작업 폴더가 같은지 확인합니다.

연결 NB01의 마지막 결과 파일이 NB02의 첫 입력입니다. 두 노트북은 변수를 자동으로 공유하지 않습니다.

다음 → 글을 숫자 목록으로 바꾸는 임베딩이 무엇인지 봅니다